



CHAPTER 2

ALGORITHMS

AMCS2034 Introduction to Data Structure & Algorithm

Overview

- ▶ Iterators
- ▶ Text Processing
 - ▶ String Operations
 - ▶ Dynamic Programming
 - ▶ Pattern Matching

Iterators

Collections

- ▶ A data structure is a collection of data organized in some fashion.
- ▶ The structure not only stores data, but also supports operations for accessing and manipulating the data.
- ▶ Without knowing data structures, you can still write programs, but your program may not be efficient.
- ▶ With a good knowledge of data structures, you can build efficient programs, which are important for practical applications.

Collections

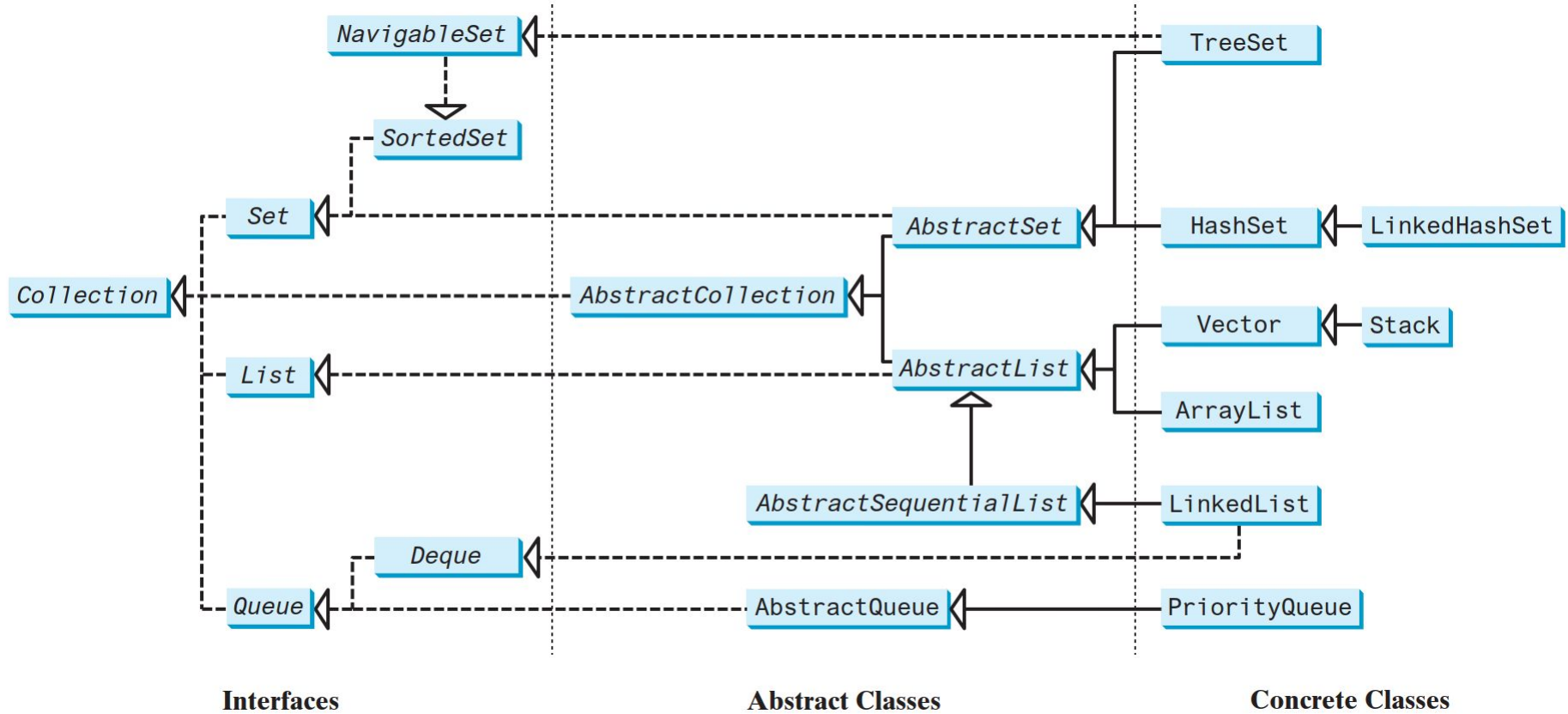
- ▶ In object-oriented thinking, a data structure, also known as a container or container object, is an object that stores other objects, referred to as data or elements.
- ▶ To define a data structure is essentially to define a class.
- ▶ The class for a data structure should use data fields to store data and provide methods to support such operations as search, insertion, and deletion.
- ▶ To create a data structure is therefore to create an instance from the class.
- ▶ You can then apply the methods on the instance to manipulate the data structure, such as inserting an element into or deleting an element from the data structure.

Collections

- ▶ The Collection interface defines the common operations for lists, vectors, stacks, queues, priority queues, and sets.
- ▶ The Java Collections Framework supports two types of containers:
 - ▶ One for storing a collection of elements is simply called a collection.
 - ▶ The other, for storing key/value pairs, is called a map.
- ▶ Maps are efficient data structures for quickly searching an element using a key.

Collections

- ▶ **Sets** store a group of non-duplicate elements.
- ▶ **Lists** store an ordered collection of elements.
- ▶ **Stacks** store objects that are processed in a last-in, first-out fashion.
- ▶ **Queues** store objects that are processed in a first-in, first-out fashion.
- ▶ **PriorityQueues** store objects that are processed in the order of their priorities.



A collection is a container that stores objects.

Iterators

- ▶ Each collection is Iterable.
- ▶ You can obtain its Iterator object to traverse all the elements in the collection.
- ▶ Iterator is a classic *design pattern* for walking through a data structure without having to expose the details of how data is stored in the data structure.

Iterators

- ▶ The Collection interface extends the Iterable interface.
- ▶ The Iterable interface defines the iterator method, which returns an iterator.
- ▶ The Iterator interface provides a uniform way for traversing elements in various types of collections.

Iterators

- ▶ The `iterator()` method in the `Iterable` interface returns an instance of `Iterator`
- ▶ which provides sequential access to the elements in the collection using the `next()` method.
- ▶ You can also use the `hasNext()` method to check whether there are more elements in the iterator,
- ▶ and the `remove()` method to remove the last element returned by the iterator.

Design Patterns

- ▶ represent the best practices used by experienced object-oriented software developers.
- ▶ are solutions to general problems that software developers faced during software development.
- ▶ These solutions were obtained by trial and error by numerous software developers over quite a substantial period of time.

Gang of Four (GOF)

- ▶ In 1994, four authors Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides published a book titled Design Patterns - Elements of Reusable Object-Oriented Software
- ▶ This initiated the concept of Design Pattern in Software development.

Gang of Four (GOF)

- ▶ These authors are collectively known as Gang of Four (GOF).
- ▶ According to these authors, design patterns are primarily based on the following principles of object orientated design.
 - ▶ Program to an interface not an implementation
 - ▶ Favor object composition over inheritance

Usage of Design Pattern

- ▶ Design Patterns have two main usages in software development.
- 1. Common platform for developers
 - ▶ Design patterns provide a standard terminology and are specific to particular scenario.
 - ▶ For example, a singleton design pattern signifies use of single object so all developers familiar with single design pattern will make use of single object and they can tell each other that program is following a singleton pattern.

Usage of Design Pattern

2. Best Practices

- ▶ Design patterns have been evolved over a long period of time and they provide best solutions to certain problems faced during software development.
- ▶ Learning these patterns helps inexperienced developers to learn software design in an easy and faster way.

Types of Design Patterns

- ▶ Creational Patterns

- ▶ These design patterns provide a way to create objects while hiding the creation logic, rather than instantiating objects directly using new operator.
- ▶ This gives program more flexibility in deciding which objects need to be created for a given use case.

Types of Design Patterns

- ▶ Structural Patterns
 - ▶ These design patterns concern class and object composition.
 - ▶ Concept of inheritance is used to compose interfaces and define ways to compose objects to obtain new functionalities.
- ▶ Behavioral Patterns
 - ▶ These design patterns are specifically concerned with communication between objects.

Design Patterns - Iterator Pattern

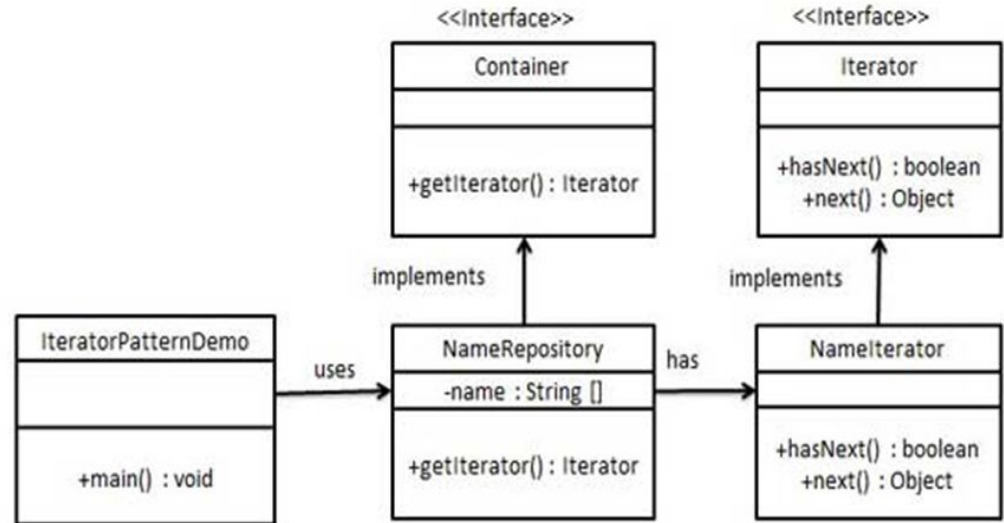
- ▶ Iterator pattern is very commonly used design pattern in the Java programming environment.
- ▶ This pattern is used to get a way to access the elements of a collection object in sequential manner without any need to know its underlying representation.
- ▶ Iterator pattern falls under behavioral pattern category.

Design Patterns - Iterator Pattern

- ▶ Implementation
 - ▶ Create a Iterator interface which narrates navigation method and a Container interface which returns the iterator.
 - ▶ Concrete classes implementing the Container interface will be responsible to implement Iterator interface and use it

Design Patterns - Iterator Pattern

- ▶ IteratorPatternDemo, uses NamesRepository, a concrete class implementation to print Names stored as a collection in NamesRepository.



Design Patterns - Iterator Pattern

- ▶ Step 1
 - ▶ Create interfaces.

Iterator.java

```
public interface Iterator {  
    public boolean hasNext();  
    public Object next();  
}
```

Design Patterns - Iterator Pattern

- ▶ Step 2
 - ▶ Create concrete class implementing the *Container* interface. This class has inner class *NameIterator* implementing the *Iterator* interface.

Design Patterns - Iterator Pattern

NameRepository.java

```
public class NameRepository implements Container {  
    public String names[] = {"Robert" , "John" , "Julie" , "Lora"};
```

```
    @Override
```

```
    public Iterator getIterator() {  
        return new NameIterator();  
    }
```

```
    private class NameIterator implements Iterator {
```

```
        int index;
```

Design Patterns - Iterator Pattern

- ▶ Step 3
 - ▶ Use the NameRepository to get iterator and print names.

IteratorPatternDemo.java

```
public class IteratorPatternDemo {  
  
    public static void main(String[] args) {  
        NameRepository namesRepository = new NameRepository();  
  
        for(Iterator iter = namesRepository.getIterator(); iter.hasNext();){  
            String name = (String)iter.next();  
            System.out.println("Name : " + name);  
        }  
    }  
}
```

Design Patterns - Iterator Pattern

- ▶ Step 4
 - ▶ Verify the output.

Name : Robert

Name : John

Name : Julie

Name : Lora

String Operations

Java Strings Class

- ▶ Strings, which are widely used in Java programming, are a sequence of characters.
- ▶ In Java programming language, strings are treated as objects.
- ▶ The Java platform provides the String class to create and manipulate strings.

Creating Strings

- ▶ The most direct way to create a string is to write –

```
String greeting = "Hello world!";
```

- ▶ Whenever it encounters a string literal in your code, the compiler creates a String object with its value in this case, "Hello world!".
- ▶ As with any other object, you can create String objects by using the new keyword and a constructor.
- ▶ The String class has 11 constructors that allow you to provide the initial value of the string using different sources, such as an array of characters.

Creating Strings

- ▶ The String class is immutable, so that once it is created a String object cannot be changed.
- ▶ If there is a necessity to make a lot of modifications to Strings of characters, then you should use String Buffer & String Builder Classes.

String Length

- ▶ Methods used to obtain information about an object are known as accessor methods.
- ▶ One accessor method that you can use with strings is the `length()` method, which returns the number of characters contained in the string object.

Concatenating Strings

- ▶ The String class includes a method for concatenating two strings –

```
string1.concat(string2);
```

- ▶ This returns a new string that is string1 with string2 added to it at the end. You can also use the concat() method with string literals, as in –

```
"My name is ".concat("Zara");
```

- ▶ Strings are more commonly concatenated with the + operator, as in –

```
"Hello," + " world" + "!"
```

- ▶ which results in –

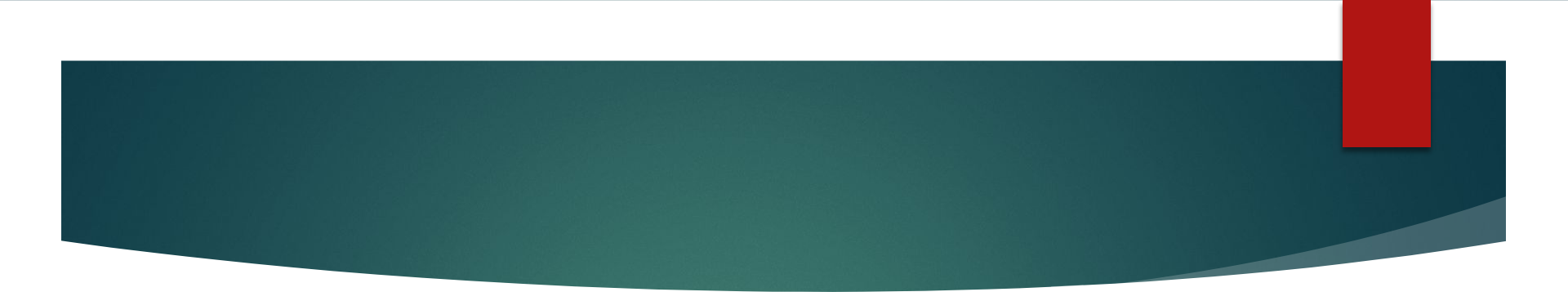
```
"Hello, world!"
```

Creating Format Strings

- ▶ You have `printf()` and `format()` methods to print output with formatted numbers.
- ▶ The `String` class has an equivalent class method, `format()`, that returns a `String` object rather than a `PrintStream` object.
- ▶ Using `String`'s static `format()` method allows you to create a formatted string that you can reuse, as opposed to a one-time print statement.

Creating Format Strings

```
System.out.printf("The value of the float variable is " +  
    "%f, while the value of the integer " +  
    "variable is %d, and the string " +  
    "is %s", floatVar, intVar, stringVar);
```



```
String fs;  
fs = String.format("The value of the float variable is " +  
    "%f, while the value of the integer " +  
    "variable is %d, and the string " +  
    "is %s", floatVar, intVar, stringVar);  
System.out.println(fs); 25/1/2022
```

Dynamic Programming

Dynamic Programming

- ▶ Dynamic programming approach is similar to divide and conquer in breaking down the problem into smaller and yet smaller possible sub-problems.
- ▶ But unlike, divide and conquer, these sub-problems are not solved independently.
- ▶ Rather, results of these smaller sub-problems are remembered and used for similar or overlapping sub-problems.

Dynamic Programming

- ▶ Dynamic programming is used where we have problems, which can be divided into similar sub-problems, so that their results can be re-used.
- ▶ Mostly, these algorithms are used for optimization.
- ▶ Before solving the in-hand sub-problem, dynamic algorithm will try to examine the results of the previously solved sub-problems.
- ▶ The solutions of sub-problems are combined in order to achieve the best solution.

Dynamic Programing

- ▶ The problem should be able to be divided into smaller overlapping sub-problem.
- ▶ An optimum solution can be achieved by using an optimum solution of smaller sub-problems.
- ▶ Dynamic algorithms use **Memoization**.

Dynamic Programming

- ▶ In contrast to *greedy algorithms*, where local optimization is addressed, dynamic algorithms are motivated for an overall optimization of the problem.
- ▶ In contrast to *divide and conquer algorithms*, where solutions are combined to achieve an overall solution, dynamic algorithms use the output of a smaller sub-problem and then try to optimize a bigger sub-problem.
- ▶ Dynamic algorithms use **Memoization** to remember the output of already solved sub-problems.

Dynamic Programming

- ▶ The following computer problems can be solved using dynamic programming approach –
 - ▶ Fibonacci number series
 - ▶ Knapsack problem
 - ▶ Tower of Hanoi
 - ▶ All pair shortest path by Floyd-Warshall
 - ▶ Shortest path by Dijkstra
 - ▶ Project scheduling

Dynamic Programming

- ▶ Dynamic programming can be used in both top-down and bottom-up manner.
- ▶ Most of the times, referring to the previous solution output is cheaper than recomputing in terms of CPU cycles.

Dynamic Programming

Those who cannot remember the past
are condemned to repeat it.

-Dynamic Programming

Dynamic Programming

- ▶ The core idea of Dynamic Programming is to avoid repeated work by remembering partial results and this concept finds its application in a lot of real life situations.

Dynamic Programming

- ▶ Writes down "1+1+1+1+1+1+1+1 =" on a sheet of paper.
- ▶ "What's that equal to?"
- ▶ Counting "Eight!"
- ▶ Writes down another "1+" on the left.
- ▶ "What about that?"
- ▶ "Nine!" " How'd you know it was nine so fast?"
- ▶ "You just added one more!"
- ▶ "So you didn't need to recount because you remembered there were eight! Dynamic Programming is just a fancy way to say remembering stuff to save time later!"

- Jonathan Poulson

Dynamic Programming

- ▶ Let's try to understand this by taking an example of Fibonacci numbers.
- ▶ $\text{Fibonacci}(n) = 1$; if $n = 0$
- ▶ $\text{Fibonacci}(n) = 1$; if $n = 1$
- ▶ $\text{Fibonacci}(n) = \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2)$
- ▶ So, the first few numbers in this series will be: 1, 1, 2, 3, 5, 8, 13, 21... and so on!

Dynamic Programming

- ▶ A code for it using pure recursion:

```
int fib (int n) {  
    if (n < 2)  
        return 1;  
    return fib(n-1) + fib(n-2);  
}
```

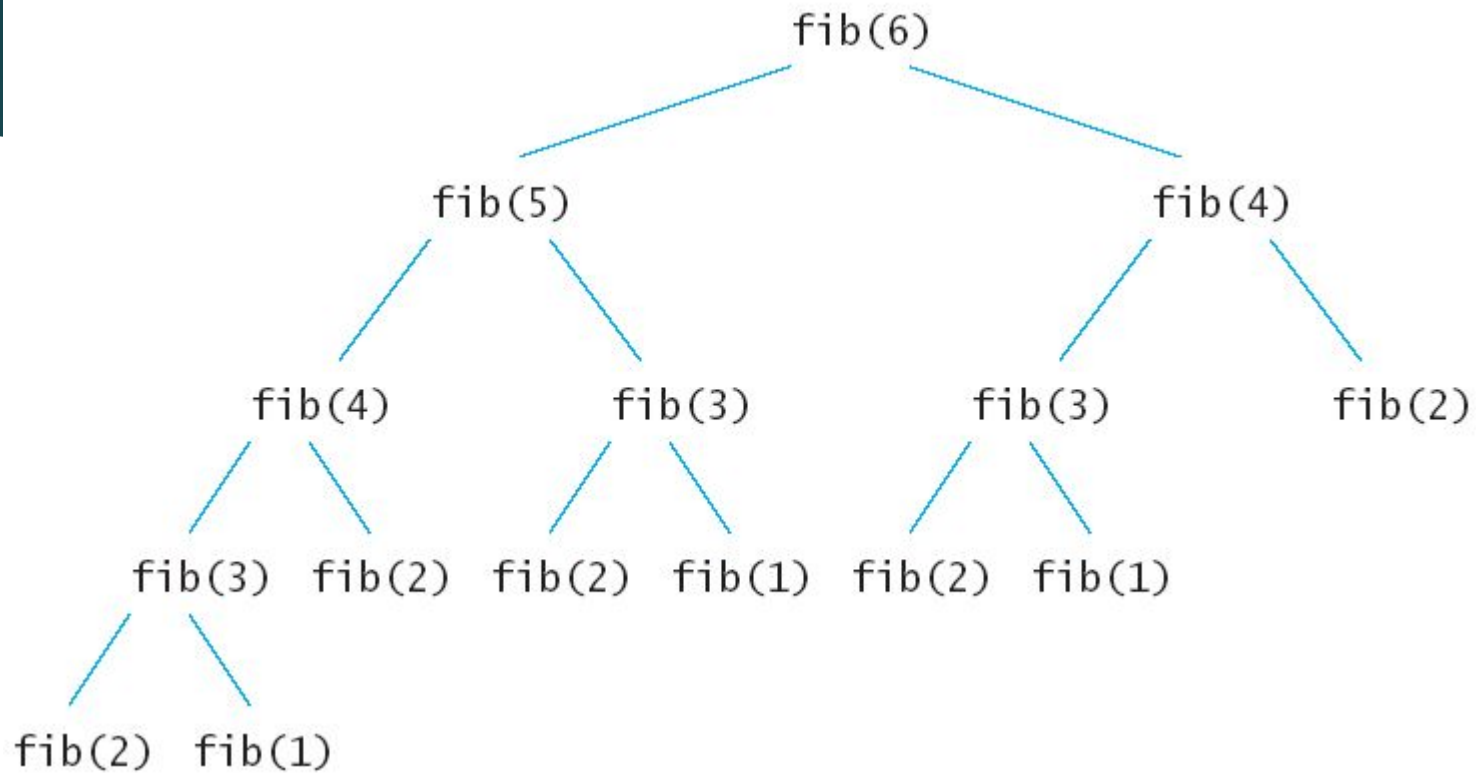
Dynamic Programming

- Using Dynamic Programming approach with memoization:

```
void fib () {  
    fibresult[0] = 1;  
    fibresult[1] = 1;  
    for (int i = 2; i < n; i++)  
        fibresult[i] = fibresult[i-1] + fibresult[i-2];  
}
```

Dynamic Programming

- ▶ In the recursive code, a lot of values are being recalculated multiple times.
- ▶ We could do good with calculating each unique quantity only once.
- ▶ Take a look at the next image to understand that how certain values were being recalculated in the recursive way:



Dynamic Programming

- ▶ Majority of the Dynamic Programming problems can be categorized into two types:
 - ▶ 1. Optimization problems.
 - ▶ 2. Combinatorial problems.
- ▶ The optimization problems expect you to select a feasible solution, so that the value of the required function is minimized or maximized.
- ▶ Combinatorial problems expect you to figure out the number of ways to do something, or the probability of some event happening.

Dynamic Programming

- ▶ Every Dynamic Programming problem has a schema to be followed:
 - ▶ Show that the problem can be broken down into optimal sub-problems.
 - ▶ Recursively define the value of the solution by expressing it in terms of optimal solutions for smaller sub-problems.
 - ▶ Compute the value of the optimal solution in bottom-up fashion.
 - ▶ Construct an optimal solution from the computed information.

Pattern Matching

Pattern Matching

- ▶ Java provides the `java.util.regex` package for pattern matching with regular expressions.
- ▶ Java regular expressions are very similar to the Perl programming language and very easy to learn.
- ▶ A regular expression is a special sequence of characters that helps you match or find other strings or sets of strings, using a specialized syntax held in a pattern.
- ▶ They can be used to search, edit, or manipulate text and data.

Pattern Matching

- ▶ The `java.util.regex` package primarily consists of the following three classes –
 - ▶ Pattern Class
 - ▶ A Pattern object is a compiled representation of a regular expression.
 - ▶ The Pattern class provides no public constructors.
 - ▶ To create a pattern, you must first invoke one of its public static `compile()` methods, which will then return a Pattern object.
 - ▶ These methods accept a regular expression as the first argument.

Pattern Matching

- ▶ **Matcher Class**
 - ▶ A Matcher object is the engine that interprets the pattern and performs match operations against an input string.
 - ▶ Like the Pattern class, Matcher defines no public constructors.
 - ▶ You obtain a Matcher object by invoking the `matcher()` method on a Pattern object.

Pattern Matching

- ▶ `PatternSyntaxException`
 - ▶ A `PatternSyntaxException` object is an unchecked exception that indicates a syntax error in a regular expression pattern.

Pattern Matching

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class Javaapp {

    public static void main(String[] args) {

        Pattern pat = Pattern.compile("JavaFX");
        Matcher mat = pat.matcher("JavaFX");

        if(mat.matches())
            System.out.println("1 : Matches");
        else
            System.out.println("1 : No Match");

        mat = pat.matcher("JavaSwing");

        if(mat.matches())
            System.out.println("2 : Matches");
        else
            System.out.println("2 : No Match");
```



END OF CHAPTER 2